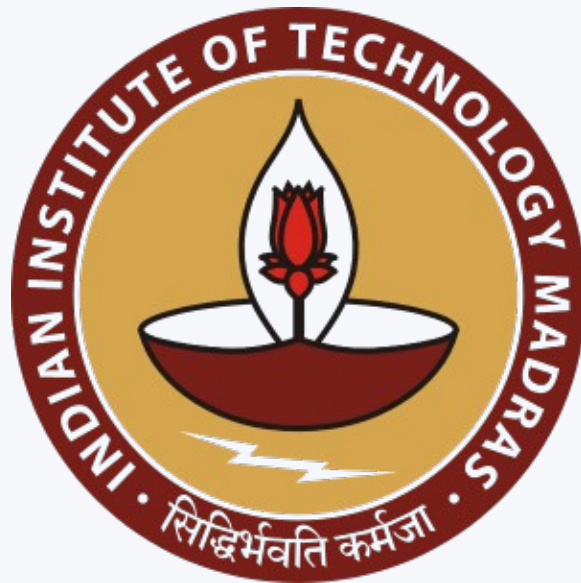


CS3205: INTRODUCTION TO COMPUTER NETWORKS

UNDER

PROF. MUKULIKA MAITY

ASSIGNMENT 3



KARTHIK KASHYAP K

EE23B030

CS3205: Introduction to Computer Networks

ASSIGNMENT 3: CONGESTION CONTROL ANALYSIS USING NS3

KARTHIK KASHYAP K

Contents

0	Introduction	2
0.1	Image and Code	2
0.2	Academic Integrity Declaration	2
1	Question 1	3
1.1	Part (a)	3
1.2	Part (b)	3
1.3	Part (c)	4
1.4	Part (d)	4
1.5	Part (e)	4
2	Question 2	5
2.1	Part (a)	6
2.2	Part (b)	6
3	Question 3	7
3.1	Part (a)	7
3.2	Part (b)	7
3.3	Part (c)	7
3.4	Part (d)	8
4	Question 4	9
4.1	Part (a)	9
4.2	Part (b)	9
5	Apologies	9

§0 Introduction

§0.1 Image and Code

Since the codes and images are best viewed externally than in a pdf file, I have also attached the respective codes and images used in this assignment in this external site: [Assignment 3 Files](#). This also contains a small amount of info that wouldnt fit into this assignment file, but need mentioning (but can otherwise be ignored).

§0.2 Academic Integrity Declaration

LLMs(namely, ChatGPT) were used in the learning for solving the questions, but every character of the codes, outputs and observations typed in the report is hand-typed by me, and not copied from elsewhere (or from someone else). Any instance where I violate this has a footnote indicating the instance and the reason for the violation.

§1 Question 1

Run the simulation for 10sec and answer the following questions. Create one TCP flow ($n_0 \rightarrow n_2$).

Solution

By changing the given code for `tcp-example-cwnd.cc` to host the network $n_0 \rightarrow r_1 \rightarrow r_2 \rightarrow n_2$, we get the following output:

```

1 [0/2] Re-checking globbed directories...
2 [2/2] Linking CXX executable ../build/scratch/ns3-dev-tcp-example-cwnd-default
3 [0/2] Re-checking globbed directories...
4 ninja: no work to do.
5
6 === Per-flow results ===
7 Flow ID: 1
8   Source      : 10.1.1.1
9   Destination : 10.1.3.2
10  Tx Bytes    : 9921620
11  Rx Bytes    : 9885332
12  Throughput  : 8.79421 Mbps
13  Lost Packets: 0
14
15 Flow ID: 2
16   Source      : 10.1.3.2
17   Destination : 10.1.1.1
18  Tx Bytes    : 195900
19  Rx Bytes    : 195536
20  Throughput  : 0.174294 Mbps
21  Lost Packets: 0
22
23 Total bytes received at sink: 9531728

```

The code for this part of the question can be found in the site [Assignment 3 Files](#).

§1.1 Part (a)

What is the maximum expected value (theoretical) of throughput (in Mbps)? Why?

Solution

Due to the bottleneck link ($r_1 \rightarrow r_2$) having a maximum throughput of 10 Mbps, the maximum achievable throughput for the given network is 10Mbps.

§1.2 Part (b)

How much is Bandwidth-Delay-Product (BDP)? Express your answer in terms of the number of packets.

Solution

We have seen in 1 (a) that the max bandwidth is 10 Mbps. The RTT is $2 \times (2 + 20 + 2) = 48$ ms.

Hence, the BDP = Bandwidth $\times$ RTT = 10Mbps $\times$ 48ms = 0.48ms = 60KB.

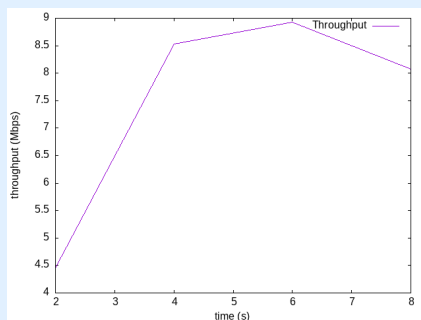
§1.3 Part (c)

What is the average computed throughput of the TCP transfer from simulation using flow-monitor? Plot throughput with time from pcap (compute throughput for each 2 sec and plot that with time).

Solution

We see from the output flow-monitor that the average computed throughput is 8.79421 Mbps. We can verify that this is correct because this is less than 10 Mbps, which is the calculated maximum throughput.

Plot¹:



§1.4 Part (d)

Is the achieved throughput approximately equal to the maximum expected value? If it is not, explain the reason for the difference.

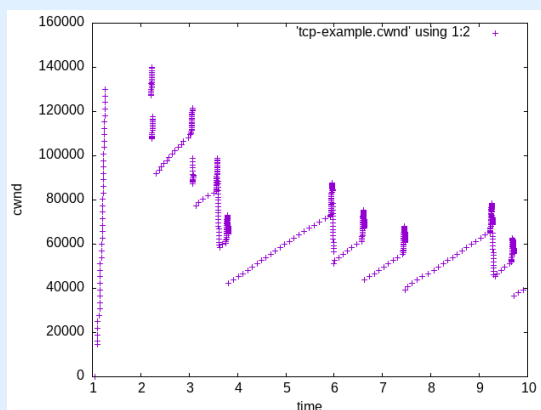
Solution

We have seen that the achieved throughput is not equal to the calculated maximum throughput because TCP is not exactly perfect (accurate). There are non idealities with respect to TCP overhead, Congestion Control Behavior and such. Along with this, there might also be inaccuracies in the **ns3** simulation itself.

§1.5 Part (e)

Plot Congestion Window (CWND) with time.

Solution



¹The code to generate the plot using the tshark data from the pcap file was written using LLMs due to lack of time.

§2 Question 2

Two Competing Flows, Add a second flow ($n_1 \rightarrow n_3$). Both share the bottleneck. Measure the throughput of each flow.

Solution

Now, but adding another topology for the network $n_1 \rightarrow r_1 \rightarrow r_2 \rightarrow n_3$ and monitoring flows for both the networks, we get the following output:

```
1 [0/2] Re-checking globbed directories...
2 [2/2] Linking CXX executable ../build/scratch/ns3-dev-tcp-example-cwnd-default
3 [0/2] Re-checking globbed directories...
4 ninja: no work to do.
5
6 === Per-flow results ===
7 Flow ID: 1
8   Source       : 10.1.1.1
9   Destination  : 10.1.3.2
10  Tx Bytes     : 4414728
11  Rx Bytes     : 4369368
12  Throughput   : 3.884 Mbps
13  Lost Packets : 0
14
15 Flow ID: 2
16   Source       : 10.1.4.2
17   Destination  : 10.1.5.2
18  Tx Bytes     : 6744684
19  Rx Bytes     : 6610548
20  Throughput   : 5.88351 Mbps
21  Lost Packets : 0
22
23 Flow ID: 3
24   Source       : 10.1.3.2
25   Destination  : 10.1.1.1
26  Tx Bytes     : 92956
27  Rx Bytes     : 92488
28  Throughput   : 0.0824402 Mbps
29  Lost Packets : 0
30
31 Flow ID: 4
32   Source       : 10.1.5.2
33   Destination  : 10.1.4.2
34  Tx Bytes     : 150076
35  Rx Bytes     : 150024
36  Throughput   : 0.13407 Mbps
37  Lost Packets : 0
38
39 Total bytes received at sink: 4205512
40 Total bytes received at sink2: 6310400
```

The code for this part of the question can be found in the site [Assignment 3 Files](#).

§2.1 Part (a)

Are the flows getting equal bandwidth?

Solution

Conceptually, we know that both the flows are supposed to get equal bandwidth, but see that in reality, the simulation gives bandwidths as 3.884 Mbps and 5.88351 Mbps, which is for the most general part in the same range.

§2.2 Part (b)

Why does TCP tend toward fair sharing?

Solution

TCP does congestion control by AIMD. So when multiple flows share a bottleneck link, and a packet loss is detected, all flows reduce their rates. This goes on until achieving the maximum bandwidth and since all of them are simultaneously occurring, they converge to have the same sending rates in the end.

But due to a few non-idealities and inaccuracies, this might vary. Here are a few reasons why it might not be exactly fair sharing:

- There is error introduced in the links, which means the packets might be getting lost for reasons other than congestion, and hence might reduce the sending speed of certain flows.
- Some flows might start earlier than others, and AIMD might result in a way where the earlier flow gets higher bandwidth because it occupies more buffer.
- When a queue is formed, there might be unsymmetrical queue in and out.
- Given that the simulation time (10s) is short, it will not result in the convergence to the fair sharing solution.

But overall, the average bandwidth is $3.884 + 5.88351 = 9.76751$ Mbps which is less than the maximum bandwidth.

§3 Question 3

Multiple Parallel Connections: create multiple flows at n_1

§3.1 Part (a)

User 1: 3 TCP flows from $n_0 \rightarrow n_2$

Solution

User 1 has 3 flows, so we need to create 2 more flows along with the existing 1 flow from $n_0 \rightarrow n_2$.

So we can just append this part into the existing flow code:

§3.2 Part (b)

User 2: 1 TCP flow (assuming from $n_1 \rightarrow n_3$)

Solution

Since there is already one flow from the previous question, there is no necessity to change anything here.

§3.3 Part (c)

Does User 1 get more bandwidth?

Solution

By making the changes from (a) and (b), we have the requirement satisfied.

By running the code using ns3, we get the following output:

```
1 [0/2] Re-checking globbed directories...
2 ninja: no work to do.
3 [0/2] Re-checking globbed directories...
4 ninja: no work to do.
5
6 === Per-flow results ===
7 Flow ID: 1
8   Source      : 10.1.1.1
9   Destination : 10.1.3.2
10  Tx Bytes    : 3124904
11  Rx Bytes    : 3064784
12  Throughput  : 2.72442 Mbps
13  Lost Packets: 0
14
15 Flow ID: 2
16   Source      : 10.1.1.1
17   Destination : 10.1.3.2
18  Tx Bytes    : 2378724
19  Rx Bytes    : 2303124
20  Throughput  : 2.08657 Mbps
21  Lost Packets: 0
22
23 Flow ID: 3
```

Solution (Contd.)

```

24 Source      : 10.1.1.1
25 Destination : 10.1.3.2
26 Tx Bytes    : 2262900
27 Rx Bytes    : 2181252
28 Throughput  : 1.96304 Mbps
29 Lost Packets: 0
30
31 Flow ID: 4
32 Source      : 10.1.4.2
33 Destination : 10.1.5.2
34 Tx Bytes    : 3596036
35 Rx Bytes    : 3458620
36 Throughput  : 3.08465 Mbps
37 Lost Packets: 0
38
39 (Flow ID: 5-8 compressed)
40
41 Total bytes received at sink: 7259612
42 Total bytes received at sink2: 3301888

```

The code for this part of the question can be found in the site [Assignment 3 Files](#).

So we see that the bandwidth of the flows are:

1. Flow 1: 2.72442 Mbps
2. Flow 2: 2.08657 Mbps
3. Flow 3: 1.96304 Mbps
4. Flow 4: 3.08465 Mbps

We see that User 1 (First 3 flows) has total bandwidth as $2.72442 + 2.08657 + 1.96304 = 6.77403$ Mbps, and User 2 (4th flow) has bandwidth as 3.08465 Mbps.

Hence, User 1 gets ~ 6.8 Mbps and User 2 gets ~ 3.1 Mbps.

Therefore: Yes, User 1 does get more bandwidth.

§3.4 Part (d)

Does TCP provide per-flow or per-user fairness?

Solution

TCP provides per-flow fairness, and not per-user fairness. This is clearly seen with the fact that User 1 got a bandwidth of ~ 6.8 Mbps and User 2 got ~ 3.1 Mbps. Hence, User 1 has ~ 2.2 times the bandwidth of User 2. Thoretically, this was supposed to be 3 times, but like we have seen in 2 (b), TCP and the simulation are not perfect, and hence there is this difference in factor.

A nice thing to note is that the total bandwidth here: $2.72442 + 2.08657 + 1.96304 + 3.08465 = 9.85868$ Mbps, which is less than the calculated maximum bandwidth possible.

§4 Question 4

TCP Variant Comparison: Repeat Q2 using two TCP variants (e.g., NewReno, CUBIC).

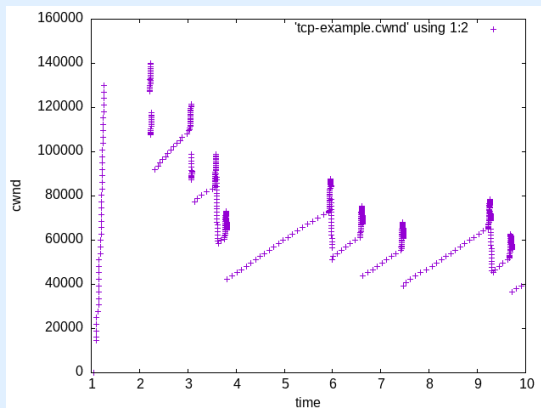
§4.1 Part (a)

Do variants behave differently? Compare their CWND using a plot.

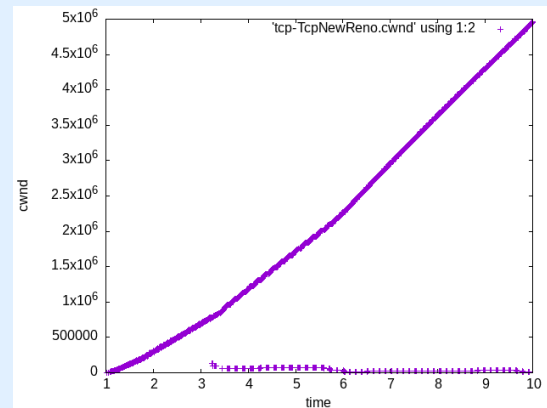
Solution

It is quite nice that the given code had a simple variable to take in the type of TCP variant that we need to use. So all I had to change for this question from the code of Q2 was to change the string value `tcp_variant` from `TcpCubic` to `TcpNewReno`.

Here are the plots of the CWND of the two variants.



TCP Cubic



TCP NewReno

Yes, TCP NewReno and TCP Cubic exhibit different congestion window behaviors. NewReno updates its congestion window more frequently as it reacts to every ACK, resulting in a dense CWND trace. Whereas, TCP Cubic updates its window based on elapsed time, producing a smoother and less frequent set of CWND updates.

§4.2 Part (b)

Which is more aggressive?

Solution

TCP Cubic is more aggressive because it increases growth using cubic rate and also recovers quickly after congestion as compared to TCP NewReno.

§5 Apologies

If this assignment seems a bit rushed, then my apologies, but I was doing this assignment when I was on a trip with my friends to Kochi (Kerala). Here are a few pictures as a proof. (I did not take much time to compile and put them here.)

More info on this can be found in the site [Assignment 3 Files](#).